EDUTECH AI PLATFORM · SERVICE RECORD

● COMPLETE · PASS

Authentication Service

auth-svc

Identity & Access Management

STATUS	TESTS	BUILD	GENERATED
Frozen	PASS Passing	SUCCESS	March 07, 2026

auth-svc — Service Completion Document

```
DOCUMENT STATUS : FROZEN
SERVICE VERSION  : 1.0.0-SNAPSHOT
FROZEN ON        : 2026-03-07
BUILD STATUS     : PASSING — 10/10 tests green
JAVA             : 17 (local) / 21 (production container)
SPRING BOOT      : 3.3.5
```

> This document is a permanent, immutable record of auth-svc as it stood at completion. It is never edited retroactively. Future changes to the service produce a new completion document at the next milestone.

Table of Contents

1. Service Purpose
2. Architectural Philosophy
3. Package Structure
4. Domain Model
5. Use Cases
6. Token Lifecycle
7. Security Architecture

8. API Contract
9. Database Schema
10. Redis Schema
11. Kafka Events
12. Infrastructure Adapters
13. Configuration Reference
14. Observability
15. Test Coverage
16. Failure Modes and Resilience
17. Dependency Inventory
18. Known Constraints and Upgrade Paths

---

1. Service Purpose

`auth-svc` is the authentication and identity boundary of the EduTech AI Platform. It is the sole service permitted to issue, rotate, or revoke tokens. Every other service validates tokens but does not produce them. This strict boundary ensures that credential logic, cryptographic material, and identity policy live in exactly one place.

Responsibilities:

RESPONSIBILITY	MECHANISM
User registration	Argon2id hashing, email OTP verification
Authentication	Captcha gating, constant-time password verification
Token issuance	JWT RS256 access token + opaque refresh token
Token rotation	Single-use, device-fingerprint-bound refresh
Session revocation	Per-token and per-user (logout all devices)
OTP delivery	Email (SMTP) and SMS (Twilio stub)
Behavioral biometrics	Keystroke dynamics risk score
Audit trail	Immutable Kafka event stream

Not a responsibility of this service:

- Authorization decisions beyond token validity (that is `@PreAuthorize` in each downstream service)
- User profile management (name changes, avatar — belongs to `parent-svc` / `center-svc`)
- Rate limiting at the infrastructure level (that is `api-gateway`)

---

2. Architectural Philosophy

2.1 Hexagonal Architecture (Ports and Adapters)

The service is structured in four concentric layers. The dependency rule is absolute: inner layers have zero knowledge of outer layers.

```
┌──────────────────────────────────────────────────────────────┐
│  api  (REST controllers, exception handler, MapStruct mapper)  │
│  ┌──────────────────────────────────────────────────────┐  │
│  │  infrastructure  (JPA, Redis, Kafka, HTTP, Security)  │  │
│  │  ┌──────────────────────────────────────────────┐  │  │
│  │  │  application  (use cases, services, DTOs, config)  │  │  │
│  │  │  ┌──────────────────────────────────────┐  │  │  │
│  │  │  │  domain  (User, Role, events, port interfaces)│  │  │  │
│  │  │  └──────────────────────────────────────┘  │  │  │
│  │  └──────────────────────────────────────────────┘  │  │
│  └──────────────────────────────────────────────────────┘  │
└──────────────────────────────────────────────────────────────┘
```

Enforcement: ArchUnit rules run on every build and will fail the build if any layer violates the dependency direction. There is no human review step required — the compiler enforces the architecture.

2.2 Design Mandates

RULE	IMPLEMENTATION
No Lombok	All constructors, getters, and builders are written explicitly
No field injection	Constructor injection only, everywhere without exception
No hardcoded values	Every configurable value flows through `${ENV_VAR}` in YAML
No setters on domain objects	State changes only through named domain methods with guards
Immutable DTOs	All DTOs are Java Records — immutable by language specification
No physical DELETE	Soft delete via `deleted_at` timestamp
No framework in domain	Domain layer imports only `jakarta.persistence` (JPA annotations) and the Java standard library

2.3 Why These Choices

Hexagonal over layered: A traditional layered architecture allows services to bypass layers (controller calling repository directly). Hexagonal architecture makes that structurally impossible because the domain knows nothing about how it is stored. Swapping PostgreSQL for any other store requires touching only one adapter class.

Records for DTOs: Immutability eliminates an entire class of bugs (accidental mutation of request objects inside service logic). The compiler guarantees it.

Argon2id over bcrypt: Argon2id won the Password Hashing Competition (2015) and is the current OWASP-recommended algorithm. It resists both GPU and ASIC attacks by requiring

configurable memory in addition to CPU time. bcrypt is memory-fixed at 4KB, which is trivially parallelisable on modern hardware.

RS256 over HS256: Asymmetric signing means that any service in the platform can validate tokens using only the public key. The private key never leaves auth-svc. With HS256, every service that validates tokens must also know the secret, expanding the attack surface.

Single-use refresh token rotation: If a refresh token is stolen and used, the legitimate user's next refresh attempt will find the token already consumed, triggering a forced re-login. Simultaneously, any stolen use will be the first use — making theft detectable. On device-fingerprint mismatch during rotation, all sessions for that user are revoked immediately.

---

3. Package Structure

```
com.edutech.auth
├── AuthSvcApplication.java                 Bootstrap — @SpringBootApplication, @ConfigurationPro
pertiesScan
│
├── domain/                                 Zero Spring dependencies. Pure Java + Jakarta Persist
ence.
│   ├── model/
│   │   ├── User.java                       Aggregate root — JPA entity, no public setters
│   │   ├── Role.java                       Enum: SUPER_ADMIN, CENTER_ADMIN, TEACHER, PARENT, STU
DENT, GUEST
│   │   └── UserStatus.java                 Enum: PENDING_VERIFICATION, ACTIVE, LOCKED, DEACTIVAT
ED
│   ├── event/                              Immutable domain events (Records) — published to Kafk
a
│   │   ├── UserRegisteredEvent.java
│   │   ├── UserAuthenticatedEvent.java     Two constructors: success (with userId) and failure
(email-only)
│   │   ├── OtpRequestedEvent.java
│   │   ├── TokenRefreshedEvent.java
│   │   └── UserLogoutEvent.java
│   └── port/
│       ├── in/                             Use case interfaces — what the application offers
│       │   ├── RegisterUserUseCase.java
│       │   ├── AuthenticateUserUseCase.java
│       │   ├── RefreshTokenUseCase.java
│       │   ├── VerifyOtpUseCase.java
│       │   └── LogoutUseCase.java
│       └── out/                            Port interfaces — what the application needs from inf
rastructure
│           ├── UserRepository.java
│           ├── PasswordHasher.java
│           ├── TokenStore.java
│           ├── OtpStore.java
```

```
│           ├── AuditEventPublisher.java
│           ├── CaptchaVerifier.java
│           ├── NotificationSender.java
│           └── AccessTokenGenerator.java
│
├── application/                          Orchestration — depends only on domain
│   ├── config/
│   │   ├── JwtProperties.java            Record: privateKeyPath, publicKeyPath, expirySeconds
(x2), issuer
│   │   └── OtpProperties.java            Record: expirySeconds, maxAttempts, length
│   ├── dto/                              All Records — immutable request/response shapes
│   │   ├── RegisterRequest.java
│   │   ├── LoginRequest.java
│   │   ├── DeviceFingerprint.java        + toFingerprintHash() → SHA-256 hex
│   │   ├── TokenPair.java
│   │   ├── StoredRefreshToken.java       + isExpired(), isSameDevice()
│   │   ├── AuthPrincipal.java            Injected via @AuthenticationPrincipal
│   │   ├── OtpSendRequest.java
│   │   ├── OtpVerifyRequest.java
│   │   ├── RefreshTokenRequest.java
│   │   ├── UserResponse.java
│   │   ├── BiometricsRequest.java
│   │   ├── KeystrokeEvent.java           + holdDuration() computed method
│   │   └── BiometricsRiskScore.java      + static factory BiometricsRiskScore.of(score, sessio
nId)
│   ├── exception/                        All extend AuthException (abstract)
│   │   ├── AuthException.java
│   │   ├── CaptchaVerificationException.java
│   │   ├── EmailAlreadyExistsException.java
│   │   ├── InvalidCredentialsException.java
│   │   ├── InvalidTokenException.java
│   │   ├── OtpExpiredException.java
│   │   ├── OtpMaxAttemptsExceededException.java
│   │   ├── AccountLockedException.java
│   │   ├── AccountNotVerifiedException.java
│   │   └── UserNotFoundException.java
│   └── service/
│       ├── UserRegistrationService.java  Implements RegisterUserUseCase
│       ├── AuthenticationService.java    Implements AuthenticateUserUseCase
│       ├── TokenRefreshService.java      Implements RefreshTokenUseCase
│       ├── LogoutService.java            Implements LogoutUseCase
│       ├── OtpService.java               Implements VerifyOtpUseCase
│       ├── TokenService.java             Shared: issue, rotate, revoke (not a use case itself)
│       └── BiometricsService.java        Keystroke dynamics risk scorer
│
├── infrastructure/                       Adapts ports to real technology
│   ├── config/
│   │   ├── Argon2Properties.java         memoryCost, iterations, parallelism, saltLength, hash
Length
│   │   ├── CaptchaProperties.java        verifyUrl, siteKey, secretKey
```

```
│   │   └── KafkaTopicProperties.java        authEvents, auditImmutable, notificationSend
│   ├── security/
│   │   ├── SecurityConfig.java              Spring Security filter chain, Argon2PasswordEncoder b
ean
│   │   ├── JwtTokenProvider.java            Implements AccessTokenGenerator — signs with RS256 pr
ivate key
│   │   ├── JwtTokenValidator.java           Validates token, returns Optional<AuthPrincipal>
│   │   ├── JwtAuthenticationFilter.java     OncePerRequestFilter — sets SecurityContext + MDC
│   │   └── Argon2PasswordHasherAdapter.java Implements PasswordHasher — wraps Argon2PasswordEncod
er
│   ├── persistence/
│   │   ├── SpringDataUserRepository.java    JPA interface (package-private) — JPQL with deleted_a
t IS NULL
│   │   └── UserPersistenceAdapter.java      Implements UserRepository — delegates to JPA
│   ├── redis/
│   │   ├── RefreshTokenRedisAdapter.java    Implements TokenStore — rt:{id} + rt:user:{userId} Se
t
│   │   └── OtpRedisAdapter.java             Implements OtpStore — otp key + attempt counter key
│   ├── kafka/
│   │   └── AuditEventKafkaAdapter.java      Implements AuditEventPublisher — async, best-effort
│   └── external/
│       ├── HCaptchaRestAdapter.java         Implements CaptchaVerifier — circuit-breaker protecte
d
│       ├── HCaptchaResponse.java            Package-private Record for hCaptcha API response
│       └── SmtpNotificationAdapter.java     Implements NotificationSender — SMTP email, Twilio SM
S stub
│
└── api/                                     HTTP boundary — depends on application ports only
    ├── AuthController.java                  /api/v1/auth/** — 7 endpoints
    ├── OtpController.java                   /api/v1/otp/** — 2 endpoints
    ├── BiometricsController.java            /api/v1/auth/biometrics — 1 endpoint
    ├── GlobalExceptionHandler.java          RFC 7807 ProblemDetail for all AuthException subtypes
    └── mapper/
        └── AuthMapper.java                  MapStruct — User → UserResponse (compile-time, zero r
eflection)
```

Total production classes: 75 Java files
Total test classes: 2 Java files
Total source files: 77

---

4. Domain Model

4.1 User Aggregate Root

The `User` class is the only JPA entity in this service. It is the aggregate root of the auth domain. All persistence and identity logic ultimately resolves to this object.

```
User
├── id           : UUID       — generated once, never updated
├── email        : String     — unique, never updated after creation
├── passwordHash : String     — Argon2id encoded, updated only via updatePassword()
├── role         : Role       — set at registration
├── status       : UserStatus — controlled via domain state machine
├── centerId     : UUID?      — null for SUPER_ADMIN and STUDENT roles
├── firstName    : String
├── lastName     : String
├── phoneNumber  : String?
├── createdAt    : Instant    — set once in constructor, never updated
├── updatedAt    : Instant    — updated by every domain method + DB trigger
├── deletedAt    : Instant?   — null = alive, non-null = soft deleted
└── version      : Long       — JPA optimistic lock counter
```

Invariants enforced in code:

- `activate()` throws `IllegalStateException` if status is not `PENDING_VERIFICATION`. A user cannot be activated twice.
- `lock()` throws `IllegalStateException` if status is `DEACTIVATED`. A deactivated account cannot be locked.
- `deactivate()` sets both `status = DEACTIVATED` and `deletedAt = Instant.now()` atomically.
- All state-changing methods update `updatedAt`. The DB trigger `trg_users_updated_at` also enforces this at the database level as a backstop.
- `User.create()` always initialises status as `PENDING_VERIFICATION`. No newly created user is ever immediately active.

4.2 Status State Machine

```
                [User.create()]
                      |
                      ▼
            PENDING_VERIFICATION
                      |
        OTP verified (activate())
                      |
                      ▼
                  ACTIVE ◄──────────────────────────────┐
                      |                                 |
            admin lock (lock())                 (future: unlock())
                      |                                 |
                      ▼                                 |
                  LOCKED ───────────────────────────────┘
                      |
            deactivate() (from ACTIVE or LOCKED)
                      |
```

```
                    ▼
                DEACTIVATED  (terminal — no transitions out)
```

4.3 Role Hierarchy

```
SUPER_ADMIN      — platform-level administration
CENTER_ADMIN     — manages one learning center (centerId required)
TEACHER          — delivers content within a center (centerId required)
PARENT           — parent of enrolled students
STUDENT          — learner (centerId required when enrolled)
GUEST            — unauthenticated access, limited API surface
```

`Role.hasHigherOrEqualRankThan(other)` — ordinal comparison, SUPER_ADMIN = highest.

---

5. Use Cases

5.1 User Registration

Entry point: `POST /api/v1/auth/register`
Implemented by: `UserRegistrationService`
Transaction scope: `@Transactional` wraps steps 3–5

```
Step 1  CAPTCHA GATE
        captchaVerifier.verify(captchaToken, ipAddress)
        → Fails fast if hCaptcha rejects or circuit breaker is open
        → Exception: CaptchaVerificationException (HTTP 422)

Step 2  EMAIL UNIQUENESS CHECK
        userRepository.existsByEmail(email)
        → Exception: EmailAlreadyExistsException (HTTP 409)

Step 3  PASSWORD HASHING
        passwordHasher.hash(password)
        → Argon2id, parameters from env vars
        → Runs synchronously — this is intentionally slow

Step 4  AGGREGATE CREATION
        User.create(email, passwordHash, role, centerId, firstName, lastName, phone)
        → Status = PENDING_VERIFICATION
        → ID = UUID.randomUUID()

Step 5  PERSISTENCE
        userRepository.save(user)
        → Flyway-managed schema, soft-delete enforced by JPQL

Step 6  AUDIT EVENT (out of transaction)
```

```
        auditEventPublisher.publish(UserRegisteredEvent)
        → Async Kafka send to audit-immutable topic
        → Failure does NOT roll back the transaction

Step 7  OTP DISPATCH
        otpService.sendOtp(email, "EMAIL_VERIFICATION", "email")
        → SecureRandom 6-digit code stored in Redis with TTL
        → Email sent via SMTP

Step 8  TOKEN PAIR ISSUANCE
        tokenService.issueTokenPair(user, deviceFingerprint)
        → Returns TokenPair to caller
        → User is registered but NOT yet active — tokens issued
           so the client can immediately call /otp/verify
```

Security note: The user receives tokens immediately upon registration. This is intentional — it allows the frontend to make an authenticated call to verify the OTP without requiring a second login step. However, all protected endpoints downstream must check that the user's status is `ACTIVE` before granting access to role-gated resources.

5.2 Authentication

Entry point: `POST /api/v1/auth/login`
Implemented by: `AuthenticationService`
Transaction scope: `@Transactional(readOnly = true)`

```
Step 1  CAPTCHA GATE
        (identical to registration — see above)

Step 2  USER LOOKUP + PASSWORD VERIFICATION (combined, constant-time)
        userRepository.findByEmail(email) → null if not found
        passwordHasher.verify(password, user.passwordHash)

        CRITICAL: If the user does not exist OR the password is wrong,
        the same exception is thrown (InvalidCredentialsException).
        This prevents email enumeration — an attacker cannot distinguish
        "email not registered" from "wrong password".

        Both failure paths publish a UserAuthenticatedEvent with
        failureReason="INVALID_CREDENTIALS" to the audit topic.

Step 3  STATUS CHECKS
        user.isLocked()              → AccountLockedException (HTTP 403)
        user.isPendingVerification() → AccountNotVerifiedException (HTTP 403)

Step 4  AUDIT EVENT
        UserAuthenticatedEvent (success variant, with userId)
```

```
Step 5  TOKEN PAIR ISSUANCE
        tokenService.issueTokenPair(user, deviceFingerprint)
```

5.3 Token Refresh

Entry point: `POST /api/v1/auth/refresh`
Implemented by: `TokenRefreshService` → delegates to `TokenService.rotateRefreshToken`

```
Step 1  LOAD STORED TOKEN
        tokenStore.find(refreshTokenId)
        → Not found → InvalidTokenException (the token was already used or never existed)

Step 2  EXPIRY CHECK
        stored.isExpired()
        → Yes → delete token + throw InvalidTokenException

Step 3  DEVICE FINGERPRINT VALIDATION
        stored.isSameDevice(incoming.toFingerprintHash())
        → Mismatch → tokenStore.deleteAllForUser(stored.userId())
                   → throw InvalidTokenException
        This is the theft-detection step. All sessions nuked on mismatch.

Step 4  USER STILL EXISTS CHECK
        userRepository.findById(stored.userId())
        → User deleted → InvalidTokenException

Step 5  OLD TOKEN DELETED BEFORE NEW ONE ISSUED
        tokenStore.delete(refreshTokenId)
        → This is the "single-use" guarantee. The window between delete
          and issue is within a single Redis pipeline call.

Step 6  NEW TOKEN PAIR ISSUED
        tokenService.issueTokenPair(user, deviceFingerprint)

Step 7  AUDIT EVENT
        TokenRefreshedEvent(userId, oldTokenId, newTokenId)
```

5.4 OTP Verification

Entry point: `POST /api/v1/otp/verify`
Implemented by: `OtpService.verifyOtp`
Transaction scope: `@Transactional`

```
Step 1  ATTEMPT COUNTER CHECK
        otpStore.getAttempts(key) >= otpProperties.maxAttempts()
        → OtpMaxAttemptsExceededException (HTTP 429)

Step 2  FETCH STORED OTP
```

```
        otpStore.find(key)
        → Not found (TTL expired) → OtpExpiredException (HTTP 410)

Step 3  CONSTANT-TIME COMPARISON
        storedOtp.equals(request.otp())
        → No match → increment attempt counter + throw OtpExpiredException
          (Same exception on wrong code as on expiry — avoids leaking state)

Step 4  SINGLE-USE CONSUMPTION
        otpStore.delete(key)

Step 5  PURPOSE-SPECIFIC ACTION
        if purpose == "EMAIL_VERIFICATION":
            user.activate()
            userRepository.save(user)
```

5.5 Logout

Single session: `POST /api/v1/auth/logout`

```
tokenStore.delete(refreshTokenId)
auditEventPublisher.publish(UserLogoutEvent(userId, tokenId, allSessions=false))
```

All sessions: `POST /api/v1/auth/logout/all`

```
tokenStore.deleteAllForUser(userId)
auditEventPublisher.publish(UserLogoutEvent(userId, tokenId=null, allSessions=true))
```

Both endpoints require a valid Bearer access token. The access token itself is not revoked (it is stateless and short-lived). Revocation of the refresh token prevents the client from obtaining new access tokens.

5.6 Behavioral Biometrics

Entry point: `POST /api/v1/auth/biometrics`
Implemented by: `BiometricsService`

Accepts a list of `KeystrokeEvent` objects (each with `keyCode`, `pressedAt`, `releasedAt`). Computes the Coefficient of Variation (CV) of hold durations as a heuristic risk score:

```
holdDuration = releasedAt - pressedAt  (for each keystroke)
mean         = sum(holdDurations) / count
stdDev       = sqrt(sum((d - mean)^2) / count)
cv           = stdDev / mean
score        = clamp(cv, 0.0, 1.0)
```

`score → 0.0` means highly consistent typing (human pattern).
`score → 1.0` means erratic timing (bot or anomalous input).

Production upgrade path: This heuristic is intentionally simple. The production replacement is a call to the Python AI sidecar in `psych-svc` / `ai-gateway-svc` once the ML model is trained on real session data.

---

6. Token Lifecycle

6.1 Access Token (JWT RS256)

```
Header  { "alg": "RS256", "typ": "JWT" }


Payload {
  "sub"      : "<userId UUID>",
  "email"    : "<user email>",
  "role"     : "<Role enum name>",
  "centerId" : "<UUID or null>",
  "deviceFP" : "<SHA-256 hex of device fingerprint>",
  "jti"      : "<UUID — unique per token>",
  "iss"      : "${JWT_ISSUER}",
  "iat"      : <issued-at epoch seconds>,
  "exp"      : <expiry epoch seconds>
}


Signature: RS256 with 2048-bit RSA private key
```

The access token is **stateless**. auth-svc does not track issued access tokens. Validation requires only the public key. Revocation is by expiry — set `JWT_ACCESS_TOKEN_EXPIRY_SECONDS` appropriately short (recommended: 900 = 15 minutes).

6.2 Refresh Token (Opaque)

The refresh token is a random UUID string. It is **not** a JWT.
The actual token data is stored in Redis under `rt:{tokenId}`.

```
StoredRefreshToken {
  userId              : UUID
  deviceFingerprintHash: String (SHA-256 hex)
  issuedAt            : Instant
  expiresAt           : Instant
}
```

The refresh token value returned to the client is only the `tokenId` (UUID string).
No sensitive data is embedded in the token itself — it is a lookup key only.

6.3 Device Fingerprint

```
raw    = userAgent + "|" + deviceId + "|" + ipSubnet
hash   = SHA-256(raw, UTF-8) → hex string
```

The hash is stored in both the access token (`deviceFP` claim) and the `StoredRefreshToken`. On refresh, the incoming fingerprint hash must match the stored hash exactly. Mismatches trigger full session revocation.

6.4 Key Material

Private key: loaded once at startup from `${JWT_PRIVATE_KEY_PATH}` (PKCS8 PEM).
Public key: loaded once at startup from `${JWT_PUBLIC_KEY_PATH}` (X.509 PEM).

Keys are loaded via `@PostConstruct`. If the file is missing or malformed, the application fails to start with `IllegalStateException`. There is no runtime reload — key rotation requires a service restart.

Generating key material:

```
# RSA 2048-bit key pair (minimum for RS256)
openssl genrsa -out private.pem 2048
openssl rsa -in private.pem -pubout -out public.pem
openssl pkcs8 -topk8 -inform PEM -outform PEM -nocrypt -in private.pem -out private_pkcs8.pem
```

---

7. Security Architecture

7.1 Threat Model

THREAT	CONTROL
Credential stuffing	hCaptcha Enterprise on every login and registration
Password brute force	Argon2id (memory-hard, slow by design) + OTP required for activation
Token theft (access)	Short expiry (configurable, default should be 15 min), RS256 signature
Token theft (refresh)	Device fingerprint binding, single-use rotation, theft detection
Session hijacking	Refresh token stolen → device mismatch → all sessions revoked
Account enumeration	Same error response for "user not found" and "wrong password"
OTP brute force	Attempt counter (max configurable), per-code TTL (configurable)
Bot registration	hCaptcha gate, email OTP verification required for activation
SQL injection	JPA parameterised queries only — no string interpolation in queries
XSS	Stateless API — no cookie-based auth, no HTML rendered

THREAT	CONTROL
CSRF	Disabled (stateless Bearer token auth has no CSRF surface)
Mass account takeover	Full session revoke on fingerprint mismatch
Replay attacks	JWT `jti` claim is unique per token; refresh tokens are single-use
Key exfiltration	Private key never transmitted over network, loaded from filesystem only
Concurrent token use	JPA `@Version` on User prevents optimistic locking race conditions
Soft-delete bypass	All JPQL queries filter `deleted_at IS NULL`
Privilege escalation	`role` is set at registration and has no mutation endpoint in this service

7.2 Password Hashing

Algorithm: **Argon2id** (hybrid of Argon2i and Argon2d — resistant to both side-channel and GPU attacks)

Parameters are configured entirely via environment variables. No defaults exist in code.
OWASP 2024 minimum recommendation for Argon2id:

```
memoryCost  ≥ 19456 KB  (19 MiB)
iterations  ≥ 2
parallelism ≥ 1
```

7.3 Security Filter Chain

```
Incoming Request
     |
     ▼
JwtAuthenticationFilter (OncePerRequestFilter)
     |
     ├─ No Authorization header? → continue (endpoint decides if auth required)
     |
     ├─ "Bearer <token>" extracted
     |
     ├─ JwtTokenValidator.validate(token)
     |    ├─ Verify RS256 signature with public key
     |    ├─ Verify issuer claim
     |    ├─ Check expiry (built into JJWT parser)
     |    └─ Extract claims → AuthPrincipal
     |
     ├─ SecurityContextHolder.setAuthentication(
     |       UsernamePasswordAuthenticationToken(principal, null, [role authority])
     |    )
     |
     └─ MDC: userId, role (propagated to all log lines for this request)
     |
```

```
      ▼
Spring Security authorization filter
      │
      ├─ Public endpoint? → permit
      ├─ Requires auth? → check SecurityContext
      └─ @PreAuthorize? → method security check
```

7.4 OWASP Top 10 Coverage

OWASP 2021	STATUS
A01 Broken Access Control	Mitigated: stateless RBAC via JWT role claim, `@EnableMethodSecurity`
A02 Cryptographic Failures	Mitigated: Argon2id, RS256, TLS enforced at gateway
A03 Injection	Mitigated: JPA parameterised queries, no raw SQL
A04 Insecure Design	Mitigated: threat-modelled token lifecycle, device binding
A05 Security Misconfiguration	Mitigated: no defaults for secrets, startup fails if config missing
A06 Vulnerable Components	Managed: Spring Boot BOM pins all transitive versions
A07 Identification and Auth Failures	Mitigated: primary concern of this service — see all above
A08 Software and Data Integrity	Managed: Maven checksum verification, pinned JJWT version
A09 Security Logging and Monitoring	Mitigated: every auth event published to immutable Kafka audit topic
A10 SSRF	Limited surface — only outbound call is to hCaptcha's fixed URL

---

8. API Contract

Base path: `/api/v1/auth`
Content-Type: `application/json`
Auth: Bearer JWT in `Authorization` header (where required)
Error format: RFC 7807 `application/problem+json`

8.1 POST /api/v1/auth/register

Auth required: No
Response on success: `201 Created`

Request body:

```
{
  "email": "student@example.com",
  "password": "s3cur3P@ssword!",
  "role": "STUDENT",
  "centerId": "a1b2c3d4-...",
  "firstName": "Priya",
```

```
  "lastName": "Sharma",
  "phoneNumber": "+919876543210",
  "captchaToken": "<hCaptcha token from frontend>",
  "deviceFingerprint": {
    "userAgent": "Mozilla/5.0 ...",
    "deviceId": "client-generated-stable-id",
    "ipSubnet": "192.168.1.0"
  }
}
```

Response body:

```
{
  "accessToken": "<JWT>",
  "refreshToken": "<UUID>",
  "accessTokenExpiresAt": "2026-03-07T13:00:00Z",
  "refreshTokenExpiresAt": "2026-04-06T12:00:00Z",
  "tokenType": "Bearer"
}
```

Error responses:

STATUS	PROBLEM TYPE	CONDITION
400	`validation-error`	Bean validation failure
409	`email-already-exists`	Email is already registered
422	`captcha-failed`	hCaptcha verification rejected

8.2 POST /api/v1/auth/login

Auth required: No
Response on success: `200 OK`

Request body:

```
{
  "email": "student@example.com",
  "password": "s3cur3P@ssword!",
  "captchaToken": "<hCaptcha token>",
  "deviceFingerprint": { ... }
}
```

Response body: Same as `/register`

Error responses:

STATUS	PROBLEM TYPE	CONDITION
401	`invalid-credentials`	Wrong email or password (intentionally ambiguous)
403	`account-locked`	Account is in LOCKED status
403	`account-not-verified`	Account is PENDING_VERIFICATION
422	`captcha-failed`	hCaptcha rejected

8.3 POST /api/v1/auth/refresh

Auth required: No (the refresh token is the credential)
Response on success: `200 OK`

Request body:

```
{
  "refreshToken": "<UUID>",
  "deviceFingerprint": { ... }
}
```

Response body: New `TokenPair`

Error responses:

STATUS	PROBLEM TYPE	CONDITION
401	`invalid-token`	Token not found, expired, or device mismatch

8.4 POST /api/v1/auth/logout

Auth required: Yes
Response on success: `204 No Content`

Request body:

```
{
  "refreshToken": "<UUID>",
  "deviceFingerprint": { ... }
}
```

8.5 POST /api/v1/auth/logout/all

Auth required: Yes
Response on success: `204 No Content`
Request body: None

Revokes all refresh tokens for the authenticated user across all devices.

8.6 GET /api/v1/auth/me

Auth required: Yes

Response on success: `200 OK`

```
{
  "id": "<UUID>",
  "email": "student@example.com",
  "firstName": "Priya",
  "lastName": "Sharma",
  "role": "STUDENT",
  "centerId": "<UUID>",
  "status": "ACTIVE",
  "createdAt": "2026-03-07T12:00:00Z"
}
```

8.7 POST /api/v1/otp/send

Auth required: No

Response on success: `202 Accepted`

```
{
  "email": "student@example.com",
  "purpose": "EMAIL_VERIFICATION",
  "channel": "email"
}
```

`purpose` values: `EMAIL_VERIFICATION`, `PASSWORD_RESET`

`channel` values: `email`, `sms`

8.8 POST /api/v1/otp/verify

Auth required: No

Response on success: `204 No Content`

```
{
  "email": "student@example.com",
  "otp": "483921",
  "purpose": "EMAIL_VERIFICATION"
}
```

Error responses:

STATUS	PROBLEM TYPE	CONDITION
410	`otp-expired`	OTP not found (TTL elapsed) or wrong code
429	`otp-max-attempts`	Max verification attempts exceeded

8.9 POST /api/v1/auth/biometrics

Auth required: No
Response on success: `200 OK`

Request body:

```
{
  "sessionId": "<client session UUID>",
  "keystrokes": [
    { "keyCode": 65, "pressedAt": 1700000000000, "releasedAt": 1700000000090 },
    { "keyCode": 66, "pressedAt": 1700000000200, "releasedAt": 1700000000310 }
  ]
}
```

Response body:

```
{
  "score": 0.23,
  "sessionId": "<UUID>",
  "evaluatedAt": "2026-03-07T12:00:00Z"
}
```

`score` range: `0.0` (confident human) → `1.0` (suspected bot)

8.10 Error Response Format (RFC 7807)

All errors are returned as `application/problem+json`:

```
{
  "type": "https://edutech.com/problems/invalid-credentials",
  "title": "Invalid credentials",
  "status": 401,
  "detail": "Email or password is incorrect",
  "instance": "/api/v1/auth/login"
}
```

---

9. Database Schema

DBMS: PostgreSQL (latest stable)
Schema: `auth_schema` (isolated from other services)
Migrations: Flyway, path `classpath:db/migration/auth/`

9.1 Migration History

VERSION	FILE	PURPOSE
V1	`V1__init_schema_and_extensions.sql`	Create schema, enable uuid-ossp and pgcrypto
V2	`V2__create_users_table.sql`	Users table, indexes, RLS, updated_at trigger

9.2 users Table

```
auth_schema.users

Column        Type          Nullable  Default                  Notes
_____________________________________________________________________
id            UUID          NOT NULL  gen_random_uuid()        PK
email         VARCHAR(255)  NOT NULL                           Unique
password_hash VARCHAR(255)  NOT NULL                           Argon2id encoded
role          VARCHAR(50)   NOT NULL                           CHECK constraint
status        VARCHAR(50)   NOT NULL  'PENDING_VERIFICATION'   CHECK constraint
center_id     UUID          NULL                               FK intent (not enforced — cross-s
ervice)
first_name    VARCHAR(100)  NOT NULL
last_name     VARCHAR(100)  NOT NULL
phone_number  VARCHAR(20)   NULL
created_at    TIMESTAMPTZ   NOT NULL  NOW()
updated_at    TIMESTAMPTZ   NOT NULL  NOW()                    Auto-updated by trigger
deleted_at    TIMESTAMPTZ   NULL                               Soft delete — NULL = alive
version       BIGINT        NOT NULL  0                        JPA optimistic lock
```

Constraints:

```
pk_users       PRIMARY KEY (id)
uq_users_email UNIQUE (email)
chk_users_role CHECK (role IN ('SUPER_ADMIN','CENTER_ADMIN','TEACHER','PARENT','STUDENT','GUEST'))
chk_users_status CHECK (status IN ('PENDING_VERIFICATION','ACTIVE','LOCKED','DEACTIVATED'))
```

9.3 Indexes

INDEX	TYPE	COLUMNS	PARTIAL FILTER	PURPOSE
`idx_users_email_active`	UNIQUE	email	`deleted_at IS NULL`	Login lookup, dedup on active emails
`idx_users_role`	BTREE	role	`deleted_at IS NULL`	Admin queries by role
`idx_users_center_id`	BTREE	center_id	`center_id IS NOT NULL AND deleted_at IS NULL`	Center-scoped user listing
`idx_users_status`	BTREE	status	`deleted_at IS NULL`	Admin queries by status

INDEX	TYPE	COLUMNS	PARTIAL FILTER	PURPOSE
`idx_users_created_at_brin`	BRIN	created_at	None	Time-range reporting, minimal size overhead

Why partial indexes everywhere: Soft-deleted rows are dead weight in an index. Partial indexes exclude `deleted_at IS NOT NULL` rows, keeping index size proportional to the live user count rather than the total row count.

Why BRIN for created_at: The users table is append-heavy. `created_at` values are naturally correlated with physical insertion order on the heap. BRIN exploits this physical correlation to store a range (min, max) per block, giving near-perfect selectivity for time-range queries at a fraction of BTREE's size.

9.4 Row Level Security

RLS is enabled on the `users` table. The policy `users_service_access` grants full row visibility to `current_user` (the service DB user at migration time). In production:

- The service connects as a dedicated non-superuser (e.g., `auth_svc_user`)
- The policy is `CREATE POLICY ... TO auth_svc_user USING (true)`
- A compromised service connection cannot access other services' data (other services use different DB credentials and schemas)
- A future multi-tenant policy can restrict rows by `center_id` without application changes

9.5 DB Trigger

`trg_users_updated_at` fires `BEFORE UPDATE` on every row modification. It sets `updated_at = NOW()` regardless of what the application sends. This is a backstop — the domain model already updates `updatedAt` in every domain method, so this trigger is defence in depth, not the primary mechanism.

---

10. Redis Schema

Redis is used for two purposes: ephemeral session state (refresh tokens) and OTP state. Both are TTL-based — Redis is the authoritative expiry mechanism.

10.1 Refresh Token Keys

```
Key pattern  :  rt:{tokenId}
Value type   :  String (JSON)
TTL          :  JWT_REFRESH_TOKEN_EXPIRY_SECONDS
Value schema :  {
                  "userId": "<UUID>",
                  "deviceFingerprintHash": "<SHA-256 hex>",
                  "issuedAt": "<ISO-8601>",
                  "expiresAt": "<ISO-8601>"
                }


Key pattern  :  rt:user:{userId}
```

```
Value type    :  Set<String> of tokenIds
TTL           :  JWT_REFRESH_TOKEN_EXPIRY_SECONDS (refreshed on each new token for this user)
Purpose       :  Enables O(1) logout-all — fetch set, delete each rt:{tokenId}, delete the set
```

Why two key structures: A single `rt:{tokenId}` key is sufficient for per-token operations. The `rt:user:{userId}` Set is a secondary index that makes "revoke all sessions" an atomic Redis operation rather than a full scan.

10.2 OTP Keys

```
Key pattern   :  otp:{email}:{purpose}
Value type    :  String (numeric OTP)
TTL           :  OTP_EXPIRY_SECONDS
Example key   :  otp:student@example.com:EMAIL_VERIFICATION

Key pattern   :  otp:{email}:{purpose}:att
Value type    :  String (integer counter)
TTL           :  OTP_EXPIRY_SECONDS (shared with the code key)
Purpose       :  Attempt counter — incremented on wrong code, checked before verify
```

When the OTP TTL expires, both keys disappear automatically. There is no background cleanup job required.

---

11. Kafka Events

Kafka mode: KRaft (no ZooKeeper)
Producer config: `JsonSerializer`, type headers disabled (`spring.json.add.type.headers=false`)
Delivery semantics: At-least-once (best-effort from auth-svc; consumers must be idempotent)

11.1 Topic: `${KAFKA_TOPIC_AUDIT_IMMUTABLE}`

Retention: forever (set `retention.ms=-1` on this topic).
All auth-svc domain events are published here.

11.2 Topic: `${KAFKA_TOPIC_NOTIFICATION_SEND}`

OTP delivery requests. Consumed by notification service (future — currently handled inline by `SmtpNotificationAdapter`).

11.3 Event Schemas

UserRegisteredEvent

```
{
  "eventId"   : "<UUID>",
  "userId"    : "<UUID>",
  "email"     : "student@example.com",
  "role"      : "STUDENT",
  "centerId"  : "<UUID or null>",
```

```
  "occurredAt": "2026-03-07T12:00:00Z"
}
```

UserAuthenticatedEvent (success)

```
{
  "eventId"       : "<UUID>",
  "userId"        : "<UUID>",
  "email"         : "student@example.com",
  "ipAddress"     : "203.0.113.1",
  "userAgent"     : "Mozilla/5.0 ...",
  "success"       : true,
  "failureReason" : null,
  "occurredAt"    : "2026-03-07T12:00:00Z"
}
```

UserAuthenticatedEvent (failure)

```
{
  "eventId"       : "<UUID>",
  "userId"        : null,
  "email"         : "unknown@example.com",
  "ipAddress"     : "203.0.113.1",
  "userAgent"     : "curl/7.88.0",
  "success"       : false,
  "failureReason" : "INVALID_CREDENTIALS",
  "occurredAt"    : "2026-03-07T12:00:00Z"
}
```

OtpRequestedEvent

```
{
  "eventId"   : "<UUID>",
  "email"     : "student@example.com",
  "purpose"   : "EMAIL_VERIFICATION",
  "channel"   : "email",
  "occurredAt": "2026-03-07T12:00:00Z"
}
```

TokenRefreshedEvent

```
{
  "eventId"    : "<UUID>",
  "userId"     : "<UUID>",
  "oldTokenId" : "<UUID>",
```

```
  "newTokenId" : "<UUID>",
  "occurredAt" : "2026-03-07T12:00:00Z"
}
```

UserLogoutEvent

```
{
  "eventId"    : "<UUID>",
  "userId"     : "<UUID>",
  "tokenId"    : "<UUID or null if all-sessions>",
  "allSessions": false,
  "occurredAt" : "2026-03-07T12:00:00Z"
}
```

All events use compact constructors that auto-generate `eventId = UUID.randomUUID()` and `occurredAt = Instant.now()` — callers never set these fields.

---

12. Infrastructure Adapters

PORT INTERFACE	ADAPTER CLASS	TECHNOLOGY
`AccessTokenGenerator`	`JwtTokenProvider`	JJWT 0.12.6, RSA private key
`PasswordHasher`	`Argon2PasswordHasherAdapter`	Spring Security Crypto, Argon2PasswordEncoder
`TokenStore`	`RefreshTokenRedisAdapter`	Spring Data Redis, StringRedisTemplate
`OtpStore`	`OtpRedisAdapter`	Spring Data Redis, StringRedisTemplate
`AuditEventPublisher`	`AuditEventKafkaAdapter`	Spring Kafka, KafkaTemplate
`CaptchaVerifier`	`HCaptchaRestAdapter`	Spring RestClient, Resilience4j CircuitBreaker
`NotificationSender`	`SmtpNotificationAdapter`	Spring Mail (JavaMailSender), Twilio stub
`UserRepository`	`UserPersistenceAdapter`	Spring Data JPA, PostgreSQL
Token validation (not a port)	`JwtTokenValidator`	JJWT 0.12.6, RSA public key

12.1 HCaptcha Circuit Breaker

```
Circuit breaker name: "captcha-client"
Config variables    : R4J_CB_CAPTCHA_WINDOW_SIZE
                      R4J_CB_CAPTCHA_FAILURE_THRESHOLD
                      R4J_CB_CAPTCHA_WAIT_DURATION
```

```
CLOSED  → requests pass through to hCaptcha API
OPEN    → captchaFallback() returns false → all requests rejected
HALF-OPEN → one probe request allowed

Fail-safe principle: if hCaptcha is unreachable, registration and login are blocked.
This is the correct behaviour — unverified traffic must not be accepted.
```

13. Configuration Reference

Every environment variable consumed by auth-svc. No defaults exist for security-critical values. The service will fail at startup if any required variable is absent.

Infrastructure

VARIABLE	TYPE	REQUIRED	DESCRIPTION
`POSTGRES_HOST`	string	yes	PostgreSQL hostname
`POSTGRES_PORT`	int	yes	PostgreSQL port (5432)
`AUTH_SVC_DB_NAME`	string	yes	Database name for auth-svc
`AUTH_SVC_DB_USERNAME`	string	yes	DB user (non-superuser)
`AUTH_SVC_DB_PASSWORD`	string	yes	DB password
`AUTH_SVC_DB_POOL_MAX_SIZE`	int	yes	HikariCP max connections
`AUTH_SVC_DB_POOL_MIN_IDLE`	int	yes	HikariCP min idle connections
`AUTH_SVC_DB_CONNECTION_TIMEOUT_MS`	int	yes	HikariCP connection timeout
`AUTH_SVC_DB_IDLE_TIMEOUT_MS`	int	yes	HikariCP idle timeout
`REDIS_HOST`	string	yes	Redis hostname
`REDIS_PORT`	int	yes	Redis port (6379)
`REDIS_PASSWORD`	string	yes	Redis password
`REDIS_SSL_ENABLED`	boolean	yes	`true` in production
`KAFKA_BOOTSTRAP_SERVERS`	string	yes	Kafka broker list
`AUTH_SVC_KAFKA_CONSUMER_GROUP`	string	yes	Consumer group ID

JWT

VARIABLE	TYPE	REQUIRED	DESCRIPTION	EXAMPLE
`JWT_PRIVATE_KEY_PATH`	path	yes	Absolute path to PKCS8 PEM	`/run/secrets/jwt_private.pem`

VARIABLE	TYPE	REQUIRED	DESCRIPTION	EXAMPLE
			file	
`JWT_PUBLIC_KEY_PATH`	path	yes	Absolute path to X.509 PEM file	`/run/secrets/jwt_public.pem`
`JWT_ACCESS_TOKEN_EXPIRY_SECONDS`	int	yes	Access token lifetime	`900` (15 min)
`JWT_REFRESH_TOKEN_EXPIRY_SECONDS`	int	yes	Refresh token lifetime	`2592000` (30 days)
`JWT_ISSUER`	string	yes	JWT `iss` claim value	`https://auth.edutech.com`

Argon2id

VARIABLE	TYPE	REQUIRED	OWASP 2024 MINIMUM	DESCRIPTION
`ARGON2_MEMORY_COST`	int (KB)	yes	19456	Memory used per hash
`ARGON2_ITERATIONS`	int	yes	2	CPU iteration count
`ARGON2_PARALLELISM`	int	yes	1	Thread count
`ARGON2_SALT_LENGTH`	int (bytes)	yes	16	Salt size
`ARGON2_HASH_LENGTH`	int (bytes)	yes	32	Output hash size

OTP

VARIABLE	TYPE	REQUIRED	RECOMMENDED	DESCRIPTION
`OTP_EXPIRY_SECONDS`	int	yes	300	OTP TTL (5 min)
`OTP_MAX_ATTEMPTS`	int	yes	3	Attempts before lockout
`OTP_LENGTH`	int	yes	6	Digit count

hCaptcha

VARIABLE	TYPE	REQUIRED	DESCRIPTION
`HCAPTCHA_VERIFY_URL`	URL	yes	`https://hcaptcha.com/siteverify`
`HCAPTCHA_SITE_KEY`	string	yes	Public site key
`HCAPTCHA_SECRET_KEY`	string	yes	Secret key (never sent to client)

Notifications

VARIABLE	TYPE	REQUIRED	DESCRIPTION
`MAIL_HOST`	string	yes	SMTP server hostname
`MAIL_PORT`	int	yes	SMTP port (587 for STARTTLS)
`MAIL_USERNAME`	string	yes	SMTP auth username
`MAIL_PASSWORD`	string	yes	SMTP auth password
`MAIL_FROM_ADDRESS`	string	yes	Sender address
`MAIL_FROM_NAME`	string	yes	Sender display name
`TWILIO_ACCOUNT_SID`	string	yes	Twilio account SID
`TWILIO_AUTH_TOKEN`	string	yes	Twilio auth token
`TWILIO_VERIFY_SERVICE_SID`	string	yes	Verify service SID

Kafka Topics

VARIABLE	TYPE	REQUIRED	DESCRIPTION
`KAFKA_TOPIC_AUTH_EVENTS`	string	yes	General auth domain events
`KAFKA_TOPIC_AUDIT_IMMUTABLE`	string	yes	Append-only, forever retention
`KAFKA_TOPIC_NOTIFICATION_SEND`	string	yes	Notification dispatch requests

Resilience4j

VARIABLE	TYPE	DESCRIPTION
`R4J_CB_CAPTCHA_WINDOW_SIZE`	int	Sliding window size for captcha circuit breaker
`R4J_CB_CAPTCHA_FAILURE_THRESHOLD`	int (%)	Failure rate to open circuit
`R4J_CB_CAPTCHA_WAIT_DURATION`	duration	Time in OPEN state (e.g., `10s`)
`R4J_RL_LOGIN_LIMIT`	int	Max login attempts per refresh period
`R4J_RL_LOGIN_REFRESH_PERIOD`	duration	Rate limiter refresh period
`R4J_RL_LOGIN_TIMEOUT`	duration	Max wait for a rate limit permit

Observability

VARIABLE	TYPE	DESCRIPTION
`OTEL_EXPORTER_OTLP_ENDPOINT`	URL	OpenTelemetry collector endpoint
`OTEL_SAMPLING_PROBABILITY`	double	Trace sampling rate (0.0–1.0)

VARIABLE	TYPE	DESCRIPTION
`LOG_LEVEL_ROOT`	string	Root logger level (`WARN` in prod)
`LOG_LEVEL_APP`	string	`com.edutech` logger level (`INFO` in prod)
`ACTUATOR_ENDPOINTS`	CSV	Exposed actuator endpoints
`APP_ENVIRONMENT`	string	Environment label tag on all metrics

Service Identity

VARIABLE	TYPE	DESCRIPTION
`AUTH_SVC_NAME`	string	`spring.application.name` value
`AUTH_SVC_PORT`	int	HTTP server port

---

14. Observability

14.1 Structured Logging

Every request that passes through `JwtAuthenticationFilter` enriches MDC with:
- `userId` — UUID of the authenticated user
- `role` — enum name of the user's role

These fields appear on every log line within the request scope, enabling log correlation without tracing infrastructure.

Log levels by event:

EVENT	LEVEL
JWT public/private key loaded	INFO
User registered	INFO
Successful login	INFO
All tokens revoked for user	INFO
Account activated	INFO
Failed login attempt	WARN
Device fingerprint mismatch — sessions revoked	WARN
hCaptcha circuit breaker open	ERROR
Failed to publish audit event	ERROR
JWT validation failed	DEBUG

14.2 Metrics

Spring Boot Actuator + Micrometer exposes Prometheus-format metrics at `/actuator/prometheus`. All metrics are tagged with `application` and `environment`.

Key metrics to alert on:

```
http_server_requests_seconds{uri="/api/v1/auth/login",status="401"}  → credential failures
http_server_requests_seconds{uri="/api/v1/auth/login",status="403"}  → locked accounts
resilience4j_circuitbreaker_state{name="captcha-client"}              → CB health
resilience4j_circuitbreaker_failure_rate{name="captcha-client"}       → hCaptcha error rate
spring_data_repository_invocations_seconds                            → DB performance
```

14.3 Distributed Tracing

`micrometer-tracing-bridge-otel` + `opentelemetry-exporter-otlp` (runtime scope) sends spans to the OTLP collector at `${OTEL_EXPORTER_OTLP_ENDPOINT}`. Sampling rate is `${OTEL_SAMPLING_PROBABILITY}`.

Trace IDs are included in all log output automatically via the Micrometer bridge.

14.4 Health Endpoints

```
GET /actuator/health        → liveness + readiness (DB, Redis connectivity)
GET /actuator/info          → build metadata
GET /actuator/prometheus    → metrics scrape endpoint
```

---

15. Test Coverage

Test runner: JUnit 5 via Maven Surefire
Total tests: 10
Failures: 0
Skipped: 0

15.1 ArchUnit Tests — `ArchitectureRulesTest`

Five structural rules enforced on every build:

TEST	RULE
`domain_must_not_depend_on_spring`	No Spring imports in `..domain..`
`domain_must_not_depend_on_infrastructure`	Domain is blind to infrastructure
`domain_must_not_depend_on_api`	Domain is blind to API layer
`application_must_not_depend_on_infrastructure`	Application uses only ports, not adapters
`application_must_not_depend_on_api`	Application has no HTTP awareness

These tests analyse compiled bytecode (not source) via ArchUnit's `ClassFileImporter`.
There is no way to accidentally bypass them with a comment or annotation.

15.2 Unit Tests — `AuthenticationServiceTest`

Five behavioural tests using Mockito (no Spring context loaded — pure unit tests):

TEST	SCENARIO
`authenticate_success`	Valid credentials → TokenPair returned, audit event published
`authenticate_userNotFound`	Unknown email → InvalidCredentialsException
`authenticate_wrongPassword`	Wrong password → InvalidCredentialsException (same exception as above)
`authenticate_lockedAccount`	LOCKED status → AccountLockedException
`authenticate_captchaFailed`	Captcha returns false → CaptchaVerificationException

15.3 Coverage Gaps (acknowledged, not defects)

The following are intentionally left for integration-level testing (Testcontainers),
which runs in a separate Maven phase (`failsafe`):

- Token rotation and theft detection (requires real Redis)
- OTP generation and verification (requires real Redis)
- Registration flow end-to-end (requires PostgreSQL + Redis + Kafka)
- JWT signing and validation with a real RSA key pair
- hCaptcha circuit breaker behaviour (requires WireMock stub)

---

16. Failure Modes and Resilience

DEPENDENCY	FAILURE BEHAVIOUR
PostgreSQL down	Service fails to start (DataSource healthcheck); existing requests fail with 500
Redis down	Token issuance and OTP fail; existing valid access tokens continue to work until expiry
Kafka down	Audit events are lost (best-effort); all auth operations succeed
hCaptcha API down	Circuit breaker opens; login and registration return 422 (fail-safe)
SMTP down	OTP email not delivered; user must request a new OTP
Private key file missing	Service refuses to start with `IllegalStateException`
Public key file missing	Service refuses to start with `IllegalStateException`
Concurrent token use	JPA `@Version` throws `OptimisticLockException` on conflict
OTP brute force	Attempt counter blocks after `OTP_MAX_ATTEMPTS` tries

16.1 Graceful Shutdown

`server.shutdown=graceful` is configured. On SIGTERM, Spring waits for in-flight requests to complete before closing connections. Kubernetes `preStop` hook should `sleep 5` to allow the load balancer to drain before the shutdown signal reaches the JVM.

---

17. Dependency Inventory

All versions are managed by the Spring Boot 3.3.5 BOM except where noted.

DEPENDENCY	VERSION	PURPOSE
`spring-boot-starter-web`	3.3.5	HTTP server (Tomcat)
`spring-boot-starter-security`	3.3.5	Security filter chain
`spring-boot-starter-data-jpa`	3.3.5	JPA / Hibernate
`spring-boot-starter-data-redis`	3.3.5	Redis client (Lettuce)
`spring-boot-starter-validation`	3.3.5	Bean Validation (Hibernate Validator)
`spring-boot-starter-mail`	3.3.5	JavaMailSender (SMTP)
`spring-kafka`	3.3.5	Kafka producer
`postgresql`	BOM-managed	JDBC driver
`flyway-core`	BOM-managed	Database migration
`flyway-database-postgresql`	BOM-managed	Flyway PG dialect
`jjwt-api`	0.12.6	JWT API
`jjwt-impl`	0.12.6	JWT RS256 implementation
`jjwt-jackson`	0.12.6	JWT Jackson serializer
`springdoc-openapi-starter-webmvc-ui`	2.6.0	OpenAPI / Swagger UI
`mapstruct`	1.5.5.Final	Compile-time DTO mapping
`resilience4j-spring-boot3`	BOM-managed	Circuit breaker, rate limiter
`micrometer-registry-prometheus`	BOM-managed	Prometheus metrics
`micrometer-tracing-bridge-otel`	BOM-managed	OTel trace bridge
`opentelemetry-exporter-otlp`	BOM-managed (OTel BOM)	OTLP span export
`archunit-junit5`	1.3.0	Test — architecture enforcement

---

18. Known Constraints and Upgrade Paths

18.1 Java 17 vs 21

The service is compiled with `--release 17`. `spring.threads.virtual.enabled=true` is configured in `application.yml` but virtual threads require Java 21. In the local dev environment (Java 17), this property is silently ignored. In production (temurin:21 container), virtual threads are active and eliminate the thread-per-request bottleneck.

Upgrade path: When the team standardises on Java 21 locally, remove the `<java.version>17</java.version>` override in the parent POM and the comment explaining it.

18.2 Biometrics Heuristic

`BiometricsService` uses a coefficient of variation heuristic. It is a stand-in for the production ML model. The interface is stable — the controller, DTO, and port are all already defined. Only `BiometricsService` needs to be replaced when the AI sidecar (`psych-svc`) is trained.

Upgrade path: Replace the CV calculation with an HTTP call to `${PSYCH_AI_SVC_BASE_URL}/v1/biometrics/score`, using `RestClient` with a Resilience4j circuit breaker and the same fallback (return score=0.5 on failure).

18.3 SMS Notification

`SmtpNotificationAdapter.sendOtpSms` logs the OTP at DEBUG level and does not call Twilio. Twilio credentials are accepted by the config but not wired to any REST call.

Upgrade path: Inject `TwilioRestClient` into `SmtpNotificationAdapter`, call `Verification.creator(to, "sms").create()` using the Verify API, protected by a circuit breaker.

18.4 Key Rotation

RSA key rotation requires a service restart to reload key material from the PEM files. There is no hot-reload mechanism.

Upgrade path: For zero-downtime key rotation, publish the public key as a JWKS endpoint (`/api/v1/auth/.well-known/jwks.json`) and implement a key version claim (`kid`) in the JWT header. Rotate by adding the new key to the JWKS before signing with it, then removing the old key after all old tokens have expired.

18.5 Refresh Token Atomicity

The delete-then-issue sequence in token rotation (Steps 5–6 in §5.3) is not atomic. A process crash between delete and issue results in the user losing their session. They must log in again. This is the correct trade-off — the alternative (issue-then-delete) creates a window where two valid refresh tokens exist simultaneously.

Upgrade path: Lua scripting on Redis can make the delete+write atomic. Acceptable for a future performance optimisation, not a correctness issue.

```
END OF DOCUMENT
```

```
auth-svc v1.0.0-SNAPSHOT

Frozen: 2026-03-07

Status: COMPLETE — all 10 tests pass, all layers verified by ArchUnit
```